

SMARTARCH BOARD

React + FastAPI + Supabase Edition | v3.0

AI-Powered Collaborative Whiteboard for System Design

Layer	Technology	Purpose
Frontend	React 18 + Vite + TypeScript	UI + Canvas (Fabric.js)
State	Zustand	Reactive store (React-native)
Realtime	Socket.IO	Multi-user sync
Backend	Python FastAPI + Uvicorn	REST API + WebSocket server
DB	Supabase (PostgreSQL)	Board persistence + Auth
Cache	Redis (Upstash)	Room state + PubSub
AI	Groq + Llama-3-70b	Architecture suggestions
Styling	Tailwind CSS	Perspective Design System

v3.0 React + FastAPI Edition | Cursor / Windsurf / Copilot Ready

TABLE OF CONTENTS

What's Inside

Ch 00 — Perspective Design System

- Brand tokens
- Typography scale
- Color palette
- Component rules
- Accessibility
- Anti-patterns & QA checklist

Ch 01 — Project Foundation & Setup

- Prerequisites
- Scaffold commands
- Folder structure
- tailwind.config.js
- index.css & dot-grid

Ch 02 — Canvas Engine — Fabric.js

- Why Fabric.js
- useFabric hook (React)
- CanvasBoard component
- Adding shapes
- Event lifecycle & cleanup

Ch 03 — State Management — Zustand

- Why Zustand
- useCanvasStore schema
- useRoomStore schema
- TypeScript interfaces

Ch 04 — Real-Time Collaboration

- Socket singleton
- useSocket hook
- FastAPI WebSocket handler
- Remote cursors

Ch 05 — AI Architecture Assistant

- Groq API service (Python)
- AI FastAPI route
- AISuggestionPanel UI
- Prompt engineering

Ch 06 — Smart Features

- Mess Clean-Up algorithm
- Context Layer Sidebar
- smartMeta schema
- Code snippet attachment

Ch 07 — Backend API — FastAPI

- FastAPI entry point
- Supabase Board model
- REST routes
- Redis caching
- .env template

Ch 08 — Deployment & DevOps

- Docker Compose
- Railway deploy
- Vercel frontend
- Environment variables

Ch 09 — Cursor / AI Coding Guide

- Project setup prompts
- Feature request prompts
- Debugging prompts
- Windsurf tips

CHAPTER 00

Perspective Design System

Spatial depth through isometric views, vanishing points, and layered elements — guiding attention via 3D-like realism.

0.1
Brand & Style Foundations

Visual Style

Modern, clean, high-contrast with spatial depth. The design moves beyond flat surfaces using shadow layering, subtle perspective shifts, and hierarchical elevation.

Typography Scale

Token	Size	Font	Weight	Use Case
xs	12px	Poppins	400	Captions, metadata
sm	14px	Poppins	400/500	Body secondary
base	16px	Poppins	400/600	Body primary
lg	20px	Poppins	600	Section labels
xl	24px	Oswald	500/700	Headings
2xl	32px	Oswald	700	Display / Hero
mono	14px	JetBrains Mono	400/500	Code blocks

Color Palette & Tokens

Token	Value	Swatch	Use
primary	#00BD7D		Actions, focus rings, active states
primary-hover	#009E68		Button hover, link hover
primary-light	#E6F9F2		Backgrounds, badges, chips
success	#16A34A		Confirmations, status OK
warning	#D97706		Caution states, alerts
danger	#DC2626		Errors, destructive actions
surface	#FFFFFF		Card / panel background
neutral-100	#F3F4F6		Page background, dividers
neutral-700	#374151		Body text, labels
neutral-900	#111827		Headings, icons

Spacing Scale

All spacing tokens: 4px · 8px · 12px · 16px · 24px · 32px. Use only these values — never arbitrary pixel gaps. In Tailwind: p-1 (4px), p-2 (8px), p-3 (12px), p-4 (16px), p-6 (24px), p-8 (32px).

Shadow Tokens

Token	CSS Value	Use
shadow-perspective	0 10px 15px -3px rgba(0,0,0,0.1)	Cards, panels
shadow-layered	0 20px 25px -5px rgba(0,0,0,0.1)	Modals, sidebars, elevated UI

0.2
Component Rules

Button Component

Primary button must use bg-primary text-white font-semibold rounded-lg py-2 px-4 transition-colors. On hover: bg-primary-hover. Disabled: opacity-50 cursor-not-allowed. Loading state: replace label with spinner.

```
src/components/ui/Button.tsx

// Button.tsx — React
type Variant = "primary" | "ghost" | "danger"
interface ButtonProps {
  variant?: Variant
  loading?: boolean
  disabled?: boolean
  children: React.ReactNode
  onClick?: () => void
}

export function Button({ variant="primary", loading, disabled, children, onClick }: ButtonProps) {
  const base = "font-semibold py-2 px-4 rounded-lg transition-colors"
  const variants = {
    primary: "bg-primary hover:bg-primary-hover text-white",
    ghost: "border border-neutral-100 text-neutral-700 hover:bg-neutral-100",
    danger: "bg-danger text-white hover:opacity-90",
  }
  return (
    <button className={`${base} ${variants[variant]} ${disabled||loading?"opacity-50 cursor-not-allowed":""}`}
      onClick={onClick} disabled={disabled||loading}>
      {loading ? <Spinner /> : children}
    </button>
  )
}
```

Card / Panel Component

Use bg-surface rounded-xl shadow-perspective p-4. For elevated modals/sidebars: shadow-layered. Never use drop-shadow on dark backgrounds — use shadow-none + border.

Input / Textarea

Base: border border-neutral-200 rounded-lg p-2 text-sm w-full. Focus: ring-2 ring-primary outline-none.
Error: border-danger ring-danger text-danger. Disabled: bg-neutral-100 cursor-not-allowed opacity-60.

0.3

Accessibility Requirements

- WCAG 2.2 AA compliance required for all components
- Minimum contrast ratio: 4.5:1 for text, 3:1 for UI elements
- All interactive elements must have visible focus ring (ring-2 ring-primary)
- Keyboard navigation: Tab order must follow visual reading order
- All images and icons must have aria-label or alt text
- Form inputs must have associated with htmlFor matching input id
- Loading states must announce to screen readers via aria-live="polite"
- Color must never be the only way to convey information (use icons + color)

0.4

Rules & Anti-Patterns

■ DO	■ DON'T
Use design tokens: text-neutral-900	Hardcode colors: style={{color:"#111827"}}
Use shadow-perspective / shadow-layered	Use random box-shadow values
Use spacing scale: p-4, gap-6	Use arbitrary px: p-[13px]
Use Poppins for body, Oswald for display	Mix system fonts or Google Fonts randomly
Pair every error color with icon + message	Use red color alone to indicate error
Use rounded-lg (8px) for cards/inputs	Mix rounded-md and rounded-xl inconsistently

0.5

QA Checklist (Code Review)

- 1. No hardcoded hex colors in JSX/TSX — only Tailwind token classes
- 2. All buttons have explicit hover, focus, disabled states
- 3. Contrast ratio verified with browser DevTools / axe extension
- 4. Font families match: Poppins/Oswald/JetBrains Mono only
- 5. Spacing uses only the 4/8/12/16/24/32px scale
- 6. Interactive elements reachable via keyboard (Tab + Enter/Space)
- 7. aria-label or title on all icon-only buttons
- 8. No inline style={{}} for visual properties (tokens only)
- 9. Cards use shadow-perspective, modals use shadow-layered
- 10. Empty and error states handled for all async data components

1.3

Folder Structure

Folder Structure

```

smartarch-board/
|- src/
|  |- assets/
|  |- components/
|  |  |- canvas/
|  |  |  |- CanvasBoard.tsx
|  |  |  |- CanvasControls.tsx
|  |  |  '- useFabric.ts          # React hook
|  |  |- layout/
|  |  |  |- Toolbar.tsx
|  |  |  |- Sidebar.tsx
|  |  |  '- Header.tsx
|  |  |- ui/
|  |  |  |- Button.tsx
|  |  |  '- Card.tsx
|  |  '- ai/
|  |     '- AISuggestionPanel.tsx
|  |- hooks/
|  |  |- useSocket.ts
|  |  '- useDebounce.ts
|  |- stores/
|  |  |- canvasStore.ts          # Zustand store
|  |  '- roomStore.ts           # Zustand store
|  |- services/
|  |  |- api.ts
|  |  '- socket.ts
|  |- types/index.ts
|  |- App.tsx
|  '- main.tsx
'- backend/
  |- main.py                    # FastAPI entry point
  |- routes/
  |  |- boards.py
  |  '- ai.py
  |- models/
  |  '- board.py                # Pydantic models
  |- services/
  |  |- supabase_client.py
  |  |- redis_client.py
  |  '- groq_service.py
  '- sockets/
    '- canvas_socket.py

```

1.4

tailwind.config.js — Perspective Design System

◆ NOTE

NEVER hardcode hex colors in components. Always use design tokens defined here.

tailwind.config.js

```
// tailwind.config.js
export default {
  content: ["/index.html", "/src/**/*.tsx,ts"],
  theme: {
    extend: {
      colors: {
        primary: { DEFAULT:"#00BD7D", hover:"#009E68", light:"#E6F9F2" },
        neutral: { 100:"#F3F4F6", 700:"#374151", 900:"#111827" },
        surface: "#FFFFFF",
        danger: "#DC2626",
        success: "#16A34A",
        warning: "#D97706",
      },
      fontFamily: {
        sans: ["Poppins", "sans-serif"],
        display: ["Oswald", "sans-serif"],
        mono: ["JetBrains Mono", "monospace"],
      },
      boxShadow: {
        perspective: "0 10px 15px -3px rgba(0,0,0,0.1)",
        layered: "0 20px 25px -5px rgba(0,0,0,0.1)",
      },
    },
  },
  plugins: [],
}
```

1.5**src/index.css — Google Fonts + Dot-Grid****src/index.css**

```
/* src/index.css */
@import url("https://fonts.googleapis.com/css2?family=Poppins:wght@400;500;600;700&family=Oswald:wght@500;700&family=JetBrains+Mono:wght@400;500&display=swap");

@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  body { @apply font-sans text-neutral-900 bg-neutral-100; }
}

/* Canvas dot-grid background */
.canvas-dot-grid {
  background-color: #F3F4F6;
  background-image: radial-gradient(circle, #D1D5DB 1px, transparent 1px);
  background-size: 24px 24px;
}
```

CHAPTER 02

Canvas Engine — Fabric.js

The heart of SmartArch. Every diagram lives here.

2.1

Why Fabric.js?

- Built-in object model (Rect, Circle, Path, Group) — no manual hit-testing needed
- Serializable JSON canvas state — perfect for Socket.IO sync
- Custom properties via `object.set(key, val)` — powers our smartMeta system
- Active selection, grouping, snapping built-in out of the box
- Works identically in React 18 — wrap in a custom hook, call in `useEffect`

2.2

useFabric.ts — React Hook

■ WARNING

Always call `canvas.dispose()` inside the `useEffect` cleanup function. Not doing so causes memory leaks and duplicate event listeners.

src/components/canvas/useFabric.ts

```
// src/components/canvas/useFabric.ts
import { useEffect, useRef } from "react"
import { fabric } from "fabric"

export function useFabric(canvasId: string) {
  const fabricRef = useRef<fabric.Canvas | null>(null)

  useEffect(() => {
    const canvas = new fabric.Canvas(canvasId, {
      width: window.innerWidth - 320,
      height: window.innerHeight - 64,
      backgroundColor: "transparent",
      selection: true,
      preserveObjectStacking: true,
    })
    fabricRef.current = canvas

    const onResize = () => {
      canvas.setDimensions({
        width: window.innerWidth - 320,
        height: window.innerHeight - 64,
      })
      canvas.renderAll()
    }
    window.addEventListener("resize", onResize)

    return () => {
      window.removeEventListener("resize", onResize)
      canvas.dispose() // <- CRITICAL cleanup
    }
  }, [])

  return fabricRef
}
```

2.3**CanvasBoard.tsx — Main Canvas Component**

src/components/canvas/CanvasBoard.tsx

```
// src/components/canvas/CanvasBoard.tsx
import { useEffect } from "react"
import { fabric } from "fabric"
import { useFabric } from "../useFabric"
import { useCanvasStore } from "../../stores/canvasStore"
import { useSocket } from "../../hooks/useSocket"

export function CanvasBoard() {
  const fabricRef = useFabric("main-canvas")
  const { setSelectedObject } = useCanvasStore()
  const { emit, on } = useSocket()

  useEffect(() => {
    const canvas = fabricRef.current
    if (!canvas) return

    const onModified = (e: fabric.IEvent) => {
      if (!e.target) return
      emit("object:updated", (e.target as any).toJSON(["smartMeta", "objectId"]))
    }
    canvas.on("object:modified", onModified)

    const cleanupRemote = on("object:updated", (data: any) => {
      fabric.util.enlivenObjects([data], (objs: fabric.Object[]) => {
        const existing = canvas.getObjects().find(
          (o: any) => o.objectId === data.objectId
        )
        if (existing) canvas.remove(existing)
        canvas.add(...objs)
        canvas.renderAll()
      }, "fabric")
    })

    canvas.on("mouse:dblclick", (e: fabric.IEvent) => {
      if (e.target) setSelectedObject(e.target)
    })

    return () => {
      canvas.off("object:modified", onModified)
      cleanupRemote?.()
    }
  }, [fabricRef.current])

  return (
    <div className="canvas-dot-grid flex-1 relative overflow-hidden">
      <canvas id="main-canvas" />
    </div>
  )
}
```

2.4**CanvasControls.ts — Shape Factories**

src/components/canvas/CanvasControls.ts

```
// src/components/canvas/CanvasControls.ts
import { fabric } from "fabric"

const genId = () => crypto.randomUUID()

export const addRect = (canvas: fabric.Canvas) => {
  const rect = new fabric.Rect({
    left: 120, top: 120, width: 160, height: 80,
    fill: "#E6F9F2", stroke: "#00BD7D", strokeWidth: 2,
    rx: 8, ry: 8,
  } as any)
  ;(rect as any).objectId = genId()
  canvas.add(rect)
  canvas.setActiveObject(rect)
  canvas.renderAll()
}

export const addCircle = (canvas: fabric.Canvas) => {
  const circle = new fabric.Circle({
    left: 200, top: 200, radius: 50,
    fill: "#EFF6FF", stroke: "#3B82F6", strokeWidth: 2,
  } as any)
  ;(circle as any).objectId = genId()
  canvas.add(circle)
  canvas.renderAll()
}

export const addText = (canvas: fabric.Canvas) => {
  const text = new fabric.IText("Service Name", {
    left: 150, top: 150,
    fontFamily: "Poppins", fontSize: 14, fill: "#111827",
  } as any)
  ;(text as any).objectId = genId()
  canvas.add(text)
  canvas.renderAll()
}
```

CHAPTER 03

State Management — Zustand

Zustand for canvas state. React Context for auth. Keep them strictly separated.

3.1

Why Zustand over Redux?

- React-native — designed for hooks, works in any component or hook
- Zero boilerplate — no actions/reducers/dispatch ceremony
- Devtools integration — first-class Redux DevTools support
- Tiny bundle (~1kb) — lighter than Redux Toolkit
- TypeScript-first — full inference without extra type ceremony
- No Provider needed — access store anywhere without wrapping

3.2

stores/canvasStore.ts

src/stores/canvasStore.ts

```
// src/stores/canvasStore.ts
import { create } from "zustand"
import { fabric } from "fabric"
import type { ObjectMeta } from "../types"

type Tool = "select" | "rect" | "circle" | "arrow" | "text"

interface CanvasStore {
  activeTool: Tool
  selectedObject: fabric.Object | null
  objectMeta: Record<string, ObjectMeta>
  setActiveTool: (tool: Tool) => void
  setSelectedObject: (obj: fabric.Object | null) => void
  setMeta: (id: string, meta: Partial<ObjectMeta>) => void
}

export const useCanvasStore = create<CanvasStore>((set, get) => ({
  activeTool: "select",
  selectedObject: null,
  objectMeta: {},
  setActiveTool: (tool) => set({ activeTool: tool }),
  setSelectedObject: (obj) => set({ selectedObject: obj }),
  setMeta: (id, meta) => set((s) => ({
    objectMeta: { ...s.objectMeta, [id]: { ...s.objectMeta[id], ...meta } }
  })),
}))
```

3.3

stores/roomStore.ts

src/stores/roomStore.ts

```
// src/stores/roomStore.ts
import { create } from "zustand"

interface User {
  id: string; name: string; color: string
  cursor: { x: number; y: number }
}

interface RoomStore {
  roomId: string | null
  users: User[]
  setRoomId: (id: string) => void
  upsertUser: (user: User) => void
  removeUser: (id: string) => void
}

export const useRoomStore = create<RoomStore>((set) => ({
  roomId: null,
  users: [],
  setRoomId: (id) => set({ roomId: id }),
  upsertUser: (user) => set((s) => {
    const idx = s.users.findIndex(u => u.id === user.id)
    if (idx >= 0) {
      const users = [...s.users]
      users[idx] = user
      return { users }
    }
    return { users: [...s.users, user] }
  }),
  removeUser: (id) => set((s) => ({
    users: s.users.filter(u => u.id !== id)
  })),
}))
```

3.4**types/index.ts — All Interfaces**

src/types/index.ts

```
// src/types/index.ts
export interface ObjectMeta {
  objectId: string
  label: string
  notes: string
  codeSnippet: string
  language: string
  links: string[]
  tags: string[]
  createdBy: string
  updatedAt: string
}

export interface Board {
  id: string
  title: string
  room_id: string
  canvas_json: string // fabric.Canvas.toJSON() output
  metadata: Record<string, ObjectMeta>
  created_at: string
  updated_at: string
}

export interface SocketEvent {
  type: "object:updated" | "cursor:moved" | "user:joined" | "user:left"
  payload: unknown
  userId: string
  roomId: string
  timestamp: number
}
```

CHAPTER 04

Real-Time Collaboration

Socket.IO + FastAPI WebSocket for seamless multi-user canvas sync.

4.1

Socket Client Singleton

■ WARNING

Create the socket ONCE as a module-level singleton. Never call `io()` inside React components — it creates multiple connections.

src/services/socket.ts

```
// src/services/socket.ts
import { io, Socket } from "socket.io-client"

let socket: Socket | null = null

export const getSocket = (): Socket => {
  if (!socket) {
    socket = io(import.meta.env.VITE_API_URL, {
      transports: ["websocket"],
      autoConnect: true,
    })
  }
  return socket
}
```

4.2

useSocket.ts — React Hook

src/hooks/useSocket.ts

```
// src/hooks/useSocket.ts
import { useCallback } from "react"
import { getSocket } from "../services/socket"
import { useRoomStore } from "../stores/roomStore"

export function useSocket() {
  const socket = getSocket()
  const roomId = useRoomStore((s) => s.roomId)

  const emit = useCallback((event: string, data: unknown) => {
    socket.emit(event, { ...(data as object), roomId })
  }, [roomId])

  const on = useCallback((event: string, handler: (d: any) => void) => {
    socket.on(event, handler)
    return () => socket.off(event, handler)
  }, [])

  return { emit, on, socket }
}
```

4.3

FastAPI WebSocket Handler

backend/sockets/canvas_socket.py

```
# backend/sockets/canvas_socket.py
import socketio
from services.redis_client import redis_client
import json

sio = socketio.AsyncServer(async_mode="asgi", cors_allowed_origins="*")

@sio.event
async def connect(sid, environ):
    print(f"Client connected: {sid}")

@sio.event
async def room_join(sid, data):
    room_id = data["roomId"]
    user = data["user"]
    await sio.enter_room(sid, room_id)
    await redis_client.setex(
        f"room:{room_id}:user:{user[\"id\"]}",
        86400, json.dumps(user)
    )
    await sio.emit("user:joined", user, room=room_id, skip_sid=sid)

@sio.event
async def object_updated(sid, data):
    await sio.emit("object:updated", data, room=data["roomId"], skip_sid=sid)

@sio.event
async def cursor_moved(sid, data):
    await sio.emit("cursor:moved", data, room=data["roomId"], skip_sid=sid)

@sio.event
async def disconnect(sid):
    await sio.emit("user:left", {"socketId": sid})
```

4.4

Remote Cursors — React

Remote Cursors – inside CanvasBoard useEffect

```
// Inside CanvasBoard.tsx useEffect, after canvas init

// Emit own cursor
const onMouseMove = (e: fabric.IEvent) => {
  const ptr = e.absolutePointer
  if (ptr) emit("cursor:moved", { x: ptr.x, y: ptr.y })
}
canvas.on("mouse:move", onMouseMove)

// Render remote cursors as DOM elements
const offRemote = on("cursor:moved", ({ userId, x, y, color }) => {
  let el = document.getElementById(`cursor-${userId}`)
  if (!el) {
    el = document.createElement("div")
    el.id = `cursor-${userId}`
    el.style.cssText = `
      position:absolute; width:10px; height:10px;
      border-radius:50%; background:${color};
      pointer-events:none; z-index:999;
      transition: transform 0.05s linear;
    `
    document.querySelector(".canvas-dot-grid")?.appendChild(el)
  }
  el.style.transform = `translate(${x}px, ${y}px)`
})
```

CHAPTER 05

AI Architecture Assistant

Groq API (Llama-3-70b) via Python for fast, context-aware suggestions.

5.1

Groq API Backend Service (Python)

`backend/services/groq_service.py`

```
# backend/services/groq_service.py
from groq import Groq
import os

client = Groq(api_key=os.getenv("GROQ_API_KEY"))

SYSTEM_PROMPT = (
    "You are an expert software architect reviewing a system design whiteboard."
    "Respond with: bottlenecks, recommendations, tech stack, scalability."
    "Be concise, technical, and actionable. Use bullet points."
)

async def get_architecture_suggestion(prompt: str) -> str:
    completion = client.chat.completions.create(
        model="llama3-70b-8192",
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": prompt}
        ],
        max_tokens=1024, temperature=0.7,
    )
    return completion.choices[0].message.content or ""
```

5.2

AI FastAPI Route

backend/routes/ai.py

```
# backend/routes/ai.py
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from services.groq_service import get_architecture_suggestion

router = APIRouter(prefix="/api/ai", tags=["ai"])

class SuggestRequest(BaseModel):
    canvasDescription: str = ""
    question: str

@router.post("/suggest")
async def suggest(body: SuggestRequest):
    if not body.question:
        raise HTTPException(status_code=400, detail="Question required")
    prompt = f"Architecture:\n{body.canvasDescription}\n\nQuestion: {body.question}"
    suggestion = await get_architecture_suggestion(prompt)
    return {"suggestion": suggestion}
```

5.3**AI Suggestion Panel.tsx — React**

src/components/ai/AISuggestionPanel.tsx

```
// src/components/ai/AISuggestionPanel.tsx
import { useState } from "react"
import axios from "axios"

interface Props { canvasJSON: string }

export function AISuggestionPanel({ canvasJSON }: Props) {
  const [question, setQuestion] = useState("")
  const [response, setResponse] = useState("")
  const [loading, setLoading] = useState(false)

  async function handleAsk() {
    if (!question.trim()) return
    setLoading(true)
    try {
      const { data } = await axios.post(
        `${import.meta.env.VITE_API_URL}/api/ai/suggest`,
        { canvasDescription: canvasJSON, question }
      )
      setResponse(data.suggestion)
    } finally { setLoading(false) }
  }

  return (
    <div className="p-4 bg-surface rounded-xl shadow-layered space-y-3">
      <h3 className="font-display text-lg text-neutral-900">AI Assistant</h3>
      <textarea rows={3}
        className="w-full border border-neutral-100 rounded-lg p-2 text-sm resize-none"
        placeholder="Ask: How should I scale this service?"
        value={question} onChange={e => setQuestion(e.target.value)} />
      <button onClick={handleAsk} disabled={loading}
        className="w-full bg-primary hover:bg-primary-hover text-white
          font-semibold py-2 rounded-lg transition-colors disabled:opacity-50">
        {loading ? "Thinking..." : "Ask AI"}
      </button>
      {response && (
        <p className="text-sm text-neutral-700 whitespace-pre-wrap">{response}</p>
      )}
    </div>
  )
}
```

5.4**Effective Prompt Templates**

- I have a monolithic e-commerce app. How should I break it into microservices?
- My Redis cache is the bottleneck. What are my options to scale it?
- I need to add real-time notifications to this design. What's the best approach?
- What database should I use for this time-series sensor data use case?

CHAPTER 06

Smart Features

Mess Clean-Up and Context Layer — the core differentiators.

6.1

Mess Clean-Up Algorithm

◆ TIP

The grid size (24px) matches the CSS dot-grid background. Objects snap exactly to dot positions.

Mess Clean-Up Algorithm

```
// src/components/canvas/CanvasControls.ts
export function messCleanUp(canvas: fabric.Canvas) {
  const GRID = 24
  let objects = canvas.getActiveObjects()
  if (objects.length === 0) {
    canvas.getObjects().forEach(obj => canvas.setActiveObject(obj))
    return messCleanUp(canvas)
  }

  const snap = (val: number) => Math.round(val / GRID) * GRID

  // Step 1: Snap each object to grid
  objects.forEach(obj => {
    obj.set({ left: snap(obj.left ?? 0), top: snap(obj.top ?? 0) })
    obj.setCoords()
  })

  // Step 2: Distribute horizontally if 3+ objects
  if (objects.length >= 3) {
    objects.sort((a, b) => (a.left ?? 0) - (b.left ?? 0))
    const startX = objects[0].left ?? 0
    objects.forEach((obj, i) => {
      obj.set({ left: snap(startX + i * 200) })
      obj.setCoords()
    })
  }
  canvas.renderAll()
}
```

6.2

Context Layer — Sidebar.tsx

src/components/layout/Sidebar.tsx

```
// src/components/layout/Sidebar.tsx
import { useCanvasStore } from "../../../stores/canvasStore"

export function Sidebar() {
  const { selectedObject, objectMeta, setMeta } = useCanvasStore()
  const id = (selectedObject as any)?.objectId ?? ""
  const meta = objectMeta[id] ?? {}

  function update(field: string, value: string) {
    setMeta(id, { [field]: value })
    ;(selectedObject as any).smartMeta = { ...meta, [field]: value }
  }

  if (!selectedObject) return null

  return (
    <aside className="fixed right-0 top-16 h-[calc(100vh-64px)] w-80
      bg-surface shadow-layered border-l border-neutral-100
      p-5 overflow-y-auto z-50">
      <h2 className="font-display text-xl text-neutral-900 mb-4">Context Layer</h2>
      <div className="space-y-4">
        <label className="block text-sm font-medium text-neutral-700">Label</label>
        <input value={meta.label ?? ""}
          onChange={e => update("label", e.target.value)}
          className="w-full border border-neutral-200 rounded-lg p-2 text-sm" />
        <label className="block text-sm font-medium text-neutral-700">Notes</label>
        <textarea value={meta.notes ?? ""}
          onChange={e => update("notes", e.target.value)}
          rows={3} className="w-full border border-neutral-200 rounded-lg p-2 text-sm" />
        <label className="block text-sm font-medium text-neutral-700">Code Snippet</label>
        <textarea value={meta.codeSnippet ?? ""}
          onChange={e => update("codeSnippet", e.target.value)}
          rows={4}
          className="w-full border border-neutral-200 rounded-lg p-2 text-sm font-mono" />
      </div>
    </aside>
  )
}
```

6.3**smartMeta Schema on Fabric Objects**

smartMeta Schema

```
// How metadata is attached to every Fabric object:
const rect = new fabric.Rect({ ... })
;(rect as any).objectId = crypto.randomUUID()
;(rect as any).smartMeta = {
  label: "API Gateway",
  notes: "Handles auth and rate limiting",
  codeSnippet: "const gateway = new Kong()",
  language: "typescript",
  links: ["https://konghq.com/docs"],
  tags: ["gateway", "auth"],
  createdBy: "user-123",
  updatedAt: new Date().toISOString(),
}

// Serialized automatically by:
canvas.toJSON(["smartMeta", "objectId"]) // <- include custom props
```

CHAPTER 07

Backend API — FastAPI

FastAPI + Supabase + Redis. REST + Socket.IO (python-socketio).

7.1

Entry Point — backend/main.py

backend/main.py

```
# backend/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
import socketio
from dotenv import load_dotenv
from routes.boards import router as boards_router
from routes.ai import router as ai_router
from sockets.canvas_socket import sio

load_dotenv()

# Create FastAPI + Socket.IO ASGI app
app = FastAPI(title="SmartArch Board API")
socket_app = socketio.ASGIApp(sio, other_asgi_app=app)

app.add_middleware(CORSMiddleware,
    allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])

app.include_router(boards_router)
app.include_router(ai_router)

# Run: uvicorn main:socket_app --reload --port 4000
```

7.2

Supabase Board Model (Pydantic)

◆ NOTE

Create a boards table in Supabase with: id (uuid), title (text), room_id (text unique), canvas_json (text), metadata (jsonb), created_at, updated_at.

backend/models/board.py

```
# backend/models/board.py
from pydantic import BaseModel
from typing import Optional, Dict, Any
from datetime import datetime

class BoardCreate(BaseModel):
    title: str = "Untitled Board"
    created_by: Optional[str] = None

class BoardUpdate(BaseModel):
    canvas_json: str
    metadata: Dict[str, Any] = {}

class BoardOut(BaseModel):
    id: str
    title: str
    room_id: str
    canvas_json: str
    metadata: Dict[str, Any]
    created_at: datetime
    updated_at: datetime
```

7.3**Board REST Routes**

backend/routes/boards.py

```
# backend/routes/boards.py
from fastapi import APIRouter, HTTPException
from services.supabase_client import supabase
from models.board import BoardCreate, BoardUpdate
from nanoid import generate

router = APIRouter(prefix="/api/boards", tags=["boards"])

@router.post("/")
async def create_board(body: BoardCreate):
    data = {**body.dict(), "room_id": generate(size=10)}
    res = supabase.table("boards").insert(data).execute()
    return res.data[0]

@router.get("/{room_id}")
async def get_board(room_id: str):
    res = supabase.table("boards").select("*") \
        .eq("room_id", room_id).single().execute()
    if not res.data:
        raise HTTPException(status_code=404, detail="Board not found")
    return res.data

@router.patch("/{room_id}/canvas")
async def update_canvas(room_id: str, body: BoardUpdate):
    res = supabase.table("boards") \
        .update({"canvas_json": body.canvas_json, "metadata": body.metadata}) \
        .eq("room_id", room_id).execute()
    return res.data[0]

@router.get("/")
async def list_boards():
    res = supabase.table("boards") \
        .select("id,title,room_id,updated_at") \
        .order("updated_at", desc=True).limit(50).execute()
    return res.data
```

7.4**Supabase & Redis Clients**

backend/services/

```
# backend/services/supabase_client.py
from supabase import create_client
import os

supabase = create_client(
    os.getenv("SUPABASE_URL"),
    os.getenv("SUPABASE_SERVICE_KEY") # Use service key on backend
)

# backend/services/redis_client.py
import redis.asyncio as aioredis
import os

redis_client = aioredis.from_url(
    os.getenv("REDIS_URL"),
    decode_responses=True
)
```

7.5**.env Files****.env Template**

```
# backend/.env
SUPABASE_URL=https://xyzabc.supabase.co
SUPABASE_SERVICE_KEY=eyJhbGciOiJI... # from Supabase project settings
REDIS_URL=redis://default:token@upstash-host.upstash.io:6379
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
PORT=4000
FRONTEND_URL=http://localhost:5173

# frontend/.env.local
VITE_API_URL=http://localhost:4000
VITE_SUPABASE_URL=https://xyzabc.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJI... # from Supabase project settings
```

CHAPTER 08

Deployment & DevOps

From localhost to production in under an hour.

8.1

Docker Compose (Full Stack)

`docker-compose.yml`

```
# docker-compose.yml (project root)
version: "3.9"
services:
  frontend:
    build:
      context: .
      dockerfile: Dockerfile.frontend
    ports: ["5173:5173"]
    environment:
      VITE_API_URL: http://backend:4000
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile.backend
    ports: ["4000:4000"]
    env_file: ./backend/.env
    command: uvicorn main:socket_app --host 0.0.0.0 --port 4000

# Dockerfile.frontend
# FROM node:20-alpine
# WORKDIR /app
# COPY . .
# RUN npm install -g pnpm && pnpm install && pnpm build
# RUN npm install -g serve
# CMD serve -s dist -l 5173

# backend/Dockerfile
# FROM python:3.11-slim
# WORKDIR /app
# COPY requirements.txt .
# RUN pip install -r requirements.txt
# COPY . .
# CMD uvicorn main:socket_app --host 0.0.0.0 --port 4000
```

8.2

Deploy to Railway (Backend)

- Push backend/ folder to a separate GitHub repository (or monorepo)
- Go to railway.app → New Project → Deploy from GitHub
- Set Start Command: `uvicorn main:socket_app --host 0.0.0.0 --port $PORT`
- Add all backend `.env` variables in Railway dashboard
- Railway auto-detects Python and builds; copy the generated public URL
- Update `VITE_API_URL` in frontend `.env.local` with the Railway URL

8.3

Deploy Frontend to Vercel

- Push frontend code to GitHub
- Go to vercel.com → New Project → Import GitHub repo
- Set Root Directory to / (project root, where package.json is)
- Add VITE_API_URL env variable pointing to Railway backend URL
- Add VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY env variables
- Deploy — Vercel handles Vite build automatically

◆ TIP

Use Supabase free tier (500MB DB, 1GB storage) and Upstash Redis free tier in production — both are generous for MVP stage.

CHAPTER 09

Cursor / AI Coding Guide

Use Cursor, Windsurf, or GitHub Copilot to build SmartArch 10x faster.

9.1

How to Use This Guide with AI Coding Tools

- Open ALL relevant files before prompting — AI tools use open tabs as context window
- Be specific about file paths: "In src/components/canvas/CanvasBoard.tsx line 34..."
- Reference this guide section explicitly: "Implement the pattern from Chapter 4.3"
- For new features: describe the TypeScript interface first, then ask for implementation
- Mention the stack explicitly: "This is a React 18 + Zustand + FastAPI project"

9.2

Project Setup Prompts (Paste into Cursor Chat)

Initialize Project Structure

Cursor Prompt – Initialize Project Structure

Create a Vite React 18 TypeScript project called smartarch-board. Install fabric, zustand, axios, socket.io-client

Create Canvas Hook

Cursor Prompt – Create Canvas Hook

Create useFabric.ts custom React hook that initializes Fabric.js canvas in useEffect, handles window resize, and manages canvas state

Set Up Zustand Stores

Cursor Prompt – Set Up Zustand Stores

Create canvasStore.ts with Zustand create(). State: activeTool (select/rect/circle/arrow/text), selectedObject, canvasData

FastAPI Backend

Cursor Prompt – FastAPI Backend

Create a FastAPI backend in /backend. Entry point is main.py. Mount python-socketio as ASGI app. Add CORS middleware

9.3

Debugging Prompts

Debug Prompt

Fabric.js canvas initializes but objects are not rendering. The canvas element exists in DOM. useFabric hook

Debug Prompt

Socket.IO events are received (confirmed in React DevTools) but canvas does not update. The on() listener is

Debug Prompt

Zustand store updates correctly (verified in Redux DevTools) but the React component does not re-render. Check

Debug Prompt

FastAPI WebSocket (python-socketio) connects but events are not forwarded to other clients. Check that sio.em

9.4**Feature Expansion Prompts****Undo/Redo****Feature Prompt – Undo/Redo**

Add undo/redo to SmartArch using Fabric.js canvas state snapshots. Store the last 50 JSON states in a Zustand

Auto-Save**Feature Prompt – Auto-Save**

Add auto-save functionality. Every 30 seconds, call canvas.toJSON() and PATCH /api/boards/:roomId/canvas. Sho

Supabase Auth**Feature Prompt – Supabase Auth**

Add Supabase Auth. Use the React Supabase client. Add a login page with email/password. Store the session in

Keyboard Shortcuts**Feature Prompt – Keyboard Shortcuts**

Add keyboard shortcuts: V = select, R = rect, C = circle, T = text, Delete = remove selected, Ctrl+A = select

TECH STACK SUMMARY

Layer	Technology	Purpose
Frontend	React 18 + Vite + TypeScript	SFC components + Hooks
Canvas	Fabric.js	Object model + JSON serialization
State	Zustand	React-native reactive store
Realtime	Socket.IO (client + python-socketio)	Multi-user canvas sync + cursors
Backend	Python FastAPI + Uvicorn	Async REST API + ASGI WebSocket
Database	Supabase (PostgreSQL)	Boards persistence + Auth + Storage
Cache	Redis / Upstash	Room state + PubSub
AI	Groq + Llama-3-70b	Architecture assistant
Design	Tailwind CSS + Perspective DS	Tokens, typography, spatial depth
Deploy FE	Vercel	Vite build + CDN
Deploy BE	Railway	Python container + auto-scaling

SmartArch Board v3.0 | React + FastAPI + Supabase Edition

AI-Powered Collaborative Whiteboard for System Design